Zero-Trust Cold Pages: Accelerating Distributed RDMA KV-Cache Offloading via Fused Asymmetric Quantization and Authenticated Encryption

Hakan Damar
Theseus Guard, Inc.
hakan.damar@theseusguard.com
https://github.com/Theseus-Guard/ZTCP · **DOI:** 10.5281/zenodo.23527048

October 2026

Abstract

Disaggregated Large Language Model (LLM) serving and hierarchical Key-Value (KV) cache pooling over RDMA (e.g., Mooncake, LMCache, SGLang HiCache, NVIDIA NIXL) mitigate GPU HBM capacity limits by offloading evicted KV pages to distributed DRAM and NVMe tiers. However, current distributed KV-cache architectures suffer from a dual bottleneck: (1) **bandwidth bloat**, where evicted KV pages traverse PCIe and RDMA fabrics in their uncompressed runtime precision (`BF16`/`FP16`/`FP8`) or incur costly host-side temporary buffer copies during serialization, and (2) **plaintext exposure and cross-tenant contamination**, where sensitive activation vectors leave the compute node unencrypted and vulnerable to prompt-inversion recovery, cross-request cache mixing, or remote memory poisoning.

We present **Zero-Trust Cold Pages (ZTCP)**, a storage- and transport-boundary architecture developed at **Theseus Guard** that extends the **Zero-Trust AI Data Plane (ZT-ADP)**, the **L1 Precedent Decision Cache**, and the **DREP (*Detect–Route–Enforce–Prove*)** governance lifecycle [3] to distributed RDMA KV-cache pools. ZTCP combines **RoPE-preserving asymmetric cold-page quantization** with **256-bit IETF RFC 8439 ChaCha20 stream encryption and 128-bit SipHash-2-4-128 Encrypt-then-MAC authentication** (distinct from ChaCha20-Poly1305) over the full 32-byte Associated Data (AAD) header and ordered ciphertext payload. By keeping active GPU Attention KV in its native hot precision while compressing cold pages by **2.67×–2.94×** prior to encryption, ZTCP reduces cryptographic work by over 62%. Across $N = 5$ independent OS process runs (×50 batch iterations per run) on an Apple M3 Max CPU (`Qwen/Qwen3.6-35B-A3B` GQA page geometry), our C++17 ARMv8.5-A FP16 NEON implementation achieves **2.35 ± 0.01 GB/s** single-core hot-byte egress throughput (**20.85×** vs. multi-operator Python/PyTorch), **17.47 ± 0.43 GB/s** (**6.55 ± 0.16 GB/s** wire) allocated 64-page batch egress (**23.02 ± 0.38 GB/s** hot ingress), and **26.65 ± 0.46 GB/s** hot-byte (**10.00 ± 0.17 GB/s** wire) 256-page preallocated CPU buffer peak egress (**25.84 ± 0.28 GB/s** hot peak ingress; **236.05×** descriptive tier ratio over Python and **11.32×** over single-core C++). In an analytical serial-egress 100GbE RoCEv2 model (11.2 GB/s effective goodput), our measured CPU throughput exceeds the Theorem 1 serial break-even threshold (17.93 GB/s hot / 6.73 GB/s wire) by 48.6%, reducing modeled 4,096-token prefix transfer latency across 64 layers (1,073.74 MB SI / 1,024.0 MiB) from **95.87 ms** (unencrypted raw `BF16`) to **76.28 ± 0.70 ms** (**1.26 ± 0.01×** speedup), while projecting **37.10 ms** (**2.58×**) under hypothetical GPU SRAM execution (950 GB/s).

1 Introduction

Long-context reasoning, multi-turn agentic workflows, and prefill-decode (PD) disaggregation rely heavily on reusing cached Key-Value (KV) states across requests [6, 11, 10]. Because high-bandwidth GPU memory (HBM) is scarce and costly, production inference engines increasingly adopt three-tier hierarchical KV caching (which we denote as Memory Tiers M1–M3 to distinguish them from the L1–L3 AI governance evaluation layers):

- **Tier M1 (Hot GPU HBM):** Active GPU memory executing Attention kernels in runtime precision (`BF16`, `FP16`, or `FP8`).
- **Tier M2 (Warm Host Memory):** Node-local pinned Host DRAM or NVMe SSD.
- **Tier M3 (Cold Distributed Pool):** Cross-node RDMA memory pools such as Mooncake Store [10], LMCache Multiprocess (MP) mode [1, 7], or 3FS.

1.1 Architectural Gaps in Production Systems

Recently, NVIDIA TensorRT-LLM introduced *NVFP4 Cold-Page Compression* [9] for worker-local Host and

Disk tiers, demonstrating that KV pages can be compressed strictly at the cold storage boundary without altering active GPU Attention precision, page identity, or block reuse scheduling. However, our audit of production distributed inference stacks reveals three critical architectural gaps when cold pages cross node boundaries over RDMA:

1. **Locality Limit in TensorRT-LLM:** TensorRT-LLM's cold-page codec operates exclusively on the worker-local `GPU` ↔ `Host/Disk` path; inter-node disaggregated transfers over NIXL transmit the uncompressed hot KV representation [9].

2. **Uncompressed RDMA Transport in SGLang HiCache & Mooncake:** In SGLang's `HiCacheStorage` and `MooncakeHostTensorAllocator`, host staging buffers are allocated at the exact element byte size of the hot GPU tensor. Offloading a 4,096-token prefix across 64 layers (GQA-8, `head_dim=128`, `BF16`) transmits **1,073.74 MB** (1,024.0 MiB) of raw bytes across the RDMA fabric.

3. **Host Double-Buffering & Plaintext Leakage in LMCache:** While LMCache MP mode [1] provides `SerdeL2AdapterWrapper` for L2 quantization (`fp8`, `turboquant`) and encryption (`aesgcm`), every store/load batch invokes `L1Manager.reserve_write(temp_keys)` to allocate secondary host buffers and executes quantization and CPU AES-GCM as discrete memory-bound passes. Moreover, LMCache's built-in `aesgcm` derives keys from a single fleet-wide master secret rather than per-tenant ephemeral sessions, leaving multi-tenant pools exposed to cross-request KV contamination (e.g., LMCache Issue #4385) and insider inspection.

1.2 Contributions & Scope Separation

This paper makes four contributions while explicitly separating what is **empirically measured in our open-source CPU artifact** from **target production/enterprise hardware extensions**:

1. **Formal Compress-then-Encrypt Break-Even vs. Wire-Saturation Model:** We prove that when cold-page compression ($r > 1$) is combined with authenticated encryption, serial transfer achieves net latency reduction over unencrypted raw transfer whenever hot-byte processing throughput exceeds the *break-even bound* $BW_{\text{fused,hot}} > \frac{r}{r-1} BW_{\text{net}}$, whereas full line-rate *wire saturation* requires $BW_{\text{fused,wire}} \geq BW_{\text{net}} \iff BW_{\text{fused,hot}} \geq r \cdot BW_{\text{net}}$.

2. **RoPE-Preserving Asymmetric Cold-Page Wire Format:** We design a 64-byte-aligned cold-page format (`32B AAD Header + Encrypted Payload + 16B MAC`) supporting both GQA (`Qwen/Qwen3.6-35B-A3B` profile) and single-buffer DeepSeek Multi-head Latent Attention (MLA, `DeepSeek-V4.1-Flash` profile) [5]. On synthetic RoPE-outlier KV pages (`seed=42`), preserving the 64-element RoPE sub-vector in 8-bit (`PRESERVE_FP8`) while quantizing NoPE and Value buffers to 4-bit yields **2.67**× compression with **0.9961–0.9962** simulated single-page *Attention Output Cosine Similarity* on GQA and **2.94**× compression with **0.9995** *Softmax Attention Weights Cosine Similarity* on single-buffer MLA (where $V = \text{None}$).

3. **Verified C++17 ARMv8.5-A FP16 NEON Codec (RFC 8439 ChaCha20 + SipHash-2-4-128):** We implement a unified C++17 SIMD codec that quantizes 32–64 FP16 values per vector pass (`vmaxvq_f16` group abs-max, vectorized `vdivq_f32` / scalar `FP32` reciprocal division narrowed to `FP16`, and vectorized `vmulq_f16` + `vcvtnq_s16_f16` nearest-even rounding), encrypts all quantized subvectors, per-group FP16 scales, and padding with 256-bit IETF RFC 8439 ChaCha20 (4-block transposed NEON), and authenticates the 32B header and ordered ciphertext via 128-bit SipHash-2-4-128. Across $N = 5$ independent process runs on Apple M3 Max, the codec achieves **26.65 ± 0.46 GB/s** hot-byte (**10.00 ± 0.17 GB/s** wire) peak egress and **25.84 ± 0.28 GB/s** hot-byte peak ingress into preallocated CPU buffers.

4. **Transactional Session Keyring, Zero-Fork Upstream Hooks & DREP Governance Integration:** We provide zero-copy plugin ABIs for SGLang HiCache, LMCache, and Mooncake, backed by a thread-safe, generation-bound `EphemeralSessionKeyRing` that uses HMAC-SHA256-based domain-separated per-generation key derivation and enforces atomic pre-encryption nonce reservation, monotonic per-slot freshness watermarks, and CSPRNG-salted in-process crypto-shredding, alongside architectural integration with the **Theseus Guard DREP** control plane [3, 4].

2 Threat Model & Break-Even Theorem

2.1 Zero-Trust Distributed KV Threat Model

We consider a multi-tenant LLM serving cluster where compute nodes offload cold KV pages across shared host DRAM, PCIe switches, RoCEv2/InfiniBand fabrics, and

remote Mooncake storage servers that are treated as **untrusted**. (In enterprise deployments, compute nodes additionally operate inside hardware-enforced Trusted Execution Environments such as NVIDIA H100/B200 Confidential Computing with Intel TDX or AMD SEV-SNP host enclaves; our public artifact evaluates the cryptographic codec and session state machine in user-space CPU memory.) An adversary or faulty storage controller may:

- **Passively eavesdrop (Prompt Inversion):** Read evicted KV pages or unencrypted scale factors in remote DRAM/SSD to reconstruct user prompts and confidential enterprise documents.
- **Cross-Tenant Contamination:** Return Tenant A's cached KV page to Tenant B due to hash collisions or indexing bugs.
- **Active KV Poisoning, Scale/Block Splicing & Stale Replay:** Flip bits in remote KV blobs, swap 16-byte quantized blocks, tamper with FP16 group scales, or replay an older superseded KV page version for a given token slot.
- **Long-Term Ciphertext Archival & Multi-Compartment Contagion:** In sovereign defense and multi-agency HPC environments [2], adversaries archive intercepted traffic or exploit shared cache pools. ZTCP employs full 256-bit RFC 8439 ChaCha20 session keys (derived via HMAC-SHA256-based domain-separated per-generation derivation mixed with a per-session 256-bit CSPRNG secret) and strict per-page 32-byte AAD binding; enterprise control planes can additionally pair session establishment with NIST **FIPS 203 ML-KEM** and **FIPS 204 ML-DSA** [8].

2.2 Break-Even Bound vs. Link Wire-Saturation Bound

Standard confidential computing encrypts uncompressed tensors ($r = 1$), inevitably adding latency. ZTCP compresses the cold page by factor $r = B_{\text{hot}}/B_{\text{cold}} > 1$ *first* and encrypts/authenticates only the reduced B_{cold} bytes. Crucially, one must distinguish between the **hot-byte processing throughput** $BW_{\text{fused,hot}} = B_{\text{hot}}/t_{\text{page}}$ and the **encoded wire output throughput** $BW_{\text{fused,wire}} = B_{\text{cold}}/t_{\text{page}} = BW_{\text{fused,hot}}/r$.

Theorem 1 (Serial Compress-then-Encrypt Break-Even Bound)**.** *Let B_{hot} be the byte size of a hot KV page, $r = B_{hot}/B_{cold} > 1$ be the net cold-page compression ratio (including the 32B AAD header and 16B MAC tag), BW_{net} be the effective RDMA network goodput, and $BW_{fused,hot}$ be the fused kernel throughput measured in hot KV bytes processed per second (all in consistent SI units, 1 GB/s = 10^9 B/s). In a non-overlapped serial pipeline (compute followed by wire transfer), the total transfer latency of an authenticated, encrypted ZTCP cold page (T_{ztcp}) is strictly smaller than that of an unencrypted raw KV page (T_{raw}) if and only if:*

$$BW_{fused,hot} > \tfrac{r}{r-1} BW_{net} \iff BW_{fused,wire} > \tfrac{1}{r-1} BW_{net} \tag{1}$$

Proof. Transmitting an unencrypted raw KV page over RDMA requires $T_{\text{raw}} = \frac{B_{\text{hot}}}{BW_{\text{net}}}$. In serial ZTCP execution, the page undergoes fused quantization + AEAD in time $\frac{B_{\text{hot}}}{BW_{\text{fused,hot}}}$ followed by RDMA transmission of $B_{\text{cold}} = \frac{B_{\text{hot}}}{r}$ bytes in time $\frac{B_{\text{hot}}}{r \cdot BW_{\text{net}}}$. Requiring $T_{\text{ztcp}} < T_{\text{raw}}$ yields:

$$\tfrac{B_{\text{hot}}}{BW_{\text{fused,hot}}} + \tfrac{B_{\text{hot}}}{r \cdot BW_{\text{net}}} < \tfrac{B_{\text{hot}}}{BW_{\text{net}}} \iff \tfrac{1}{BW_{\text{fused,hot}}} < \tfrac{r-1}{r \cdot BW_{\text{net}}}$$

Inverting both sides establishes Inequality (1), with net serial speedup $S = \left(\frac{1}{r} + \frac{BW_{\text{net}}}{BW_{\text{fused,hot}}}\right)^{-1}$. □

Corollary 1 (Break-Even vs. Full Link Wire Saturation): Whereas break-even requires $BW_{\text{fused,hot}} > \frac{r}{r-1} BW_{\text{net}}$, keeping an RDMA link 100% saturated with compressed cold blobs requires $BW_{\text{fused,wire}} \geq BW_{\text{net}} \iff BW_{\text{fused,hot}} \geq r \cdot BW_{\text{net}}$. For GQA cold pages with `INT4_SYM + Preserve RoPE (FP8)` ($r = 262,144/98,352 = 2.6654$, $\frac{r}{r-1} = 1.6005$):

- **100GbE RoCEv2 (BW_{net} = 11.2 GB/s):** Serial break-even requires $BW_{\text{fused,hot}} > \mathbf{17.93}$ GB/s (6.73 GB/s wire), while 100% wire saturation requires $BW_{\text{fused,hot}} \geq \mathbf{29.85}$ GB/s (11.20 GB/s wire). Our measured M3 Max CPU peak throughput (**26.65 ± 0.46 GB/s** hot / **10.00 ± 0.17 GB/s** wire on `INT4_SYM`, and **23.76 ± 0.27 GB/s** hot / **10.40 ± 0.12 GB/s** wire on `K8V4_ASYM`, exceeding its 19.92 GB/s hot break-even bound by 19.3%) exceeds the `INT4_SYM` 100GbE break-even bound by 48.6% (corresponding to an analytical ratio $BW_{\text{fused,wire}}/BW_{\text{net}}$ of 89.3%–92.9% of modeled 100GbE wire capacity from a single CPU socket—not a live NIC measurement—and yielding a $1.26 \pm 0.01\times$ analytical serial speedup over unencrypted raw `BF16`).
- **400GbE NDR InfiniBand (BW_{net} = 45.0 GB/s):** Break-even requires $BW_{\text{fused,hot}} > \mathbf{72.02}$ GB/s (27.02 GB/s wire) and full wire saturation requires $BW_{\text{fused,hot}} \geq \mathbf{119.94}$ GB/s (45.00 GB/s wire), which exceeds single-socket CPU throughput and motivates GPU SRAM execution.

3 ZTCP Architecture & Cryptographic Construction

3.1 Compact Cold-Page Wire Specification & Bijective Nonce

Each cold KV page is encoded as a contiguous, 64-byte DMA-aligned binary blob:

1. **32-Byte Plaintext Authenticated Header (AAD):**

```
struct ColdPageHeaderRaw {
  uint16_t magic;          // 0x5A54 ("ZT")
  uint8_t  version;        // 1
  uint8_t  flags;          // Preset | RoPE
  uint32_t tenant_id_hash; // BLAKE2s-32
  uint32_t model_id_hash;  // Model/Snapshot
  uint16_t layer_id;       // Layer index
  uint16_t head_group_id;  // Group index
  uint64_t page_token_hash;// BLAKE2s-64
  uint64_t sequence_nonce; // Monotonic u64
}; // 32 bytes exact
```

2. **Bijective 96-Bit ChaCha20 Nonce Packing:** Rather than XOR-mixing header fields (which would allow cross-field XOR cancellations between `sequence_nonce` and `model_id_hash`/`page_token_hash`), ZTCP packs the 96-bit (12-byte) IETF RFC 8439 ChaCha20 nonce bijectively as:

$$\texttt{nonce}_{96} = \texttt{seq_nonce}_{64} \parallel \texttt{layer_id}_{16} \parallel \texttt{head_group_id}_{16}$$

3. **Encrypted Quantized Payload & Scales:** Contains the separated Key RoPE slice (`PRESERVE_FP8`: 1B INT8 data + 2B FP16 scale per 16 elements), Key NoPE slice (packed 4-bit nibbles or 8-bit data + 2B FP16 scale per 16 elements), Value slice (packed 4-bit nibbles or 8-bit data + 2B FP16 scale per 16 elements; omitted for single-buffer DeepSeek-MLA), and deterministic zero tail padding to a 64-byte boundary. **All sections—including all FP16 scale arrays and tail padding—are encrypted under the 256-bit RFC 8439 ChaCha20 keystream.**

4. **16-Byte Authentication Tag (Encrypt-then-MAC):** Computed sequentially via standard **SipHash-2-4-128** (Aumasson–Bernstein, using two 64-bit MAC keys mk_0, mk_1 derived via domain-separated HMAC-SHA256) over the exact byte sequence `header_32b` $\parallel$ `ordered_ciphertext_payload`.

3.2 Unified Single/Batch/Peak Execution & L1 Scale Staging

All three native C++17 entry-points (`fused_encode_cold_page_cpu`, `fused_encode_cold_pages_batch_cpu`, and `fused_encode_cold_pages_peak_into_cpu`) invoke the same canonical single-page encoder (`encode_one_cold_page_into_cpu`), guaranteeing bit-for-bit identical wire output (`single == batch == peak`):

1. **Multi-Group FP16 Vector Quantization (32–64 Elements):** On ARMv8.5-A NEON (`csrc/fused_cold_page_cpu.cpp`), 128-bit registers load 32 or 64 contiguous `FP16` values (two or four 16-element groups) and compute group abs-maxima via `vmaxvq_f16`. Each maximum is widened to `FP32` (`float`) to compute the group scale (`fmax * (1.0f / 7.0f)` or `1.0f / 127.0f`, stored as `FP16`) and its exact IEEE-754 `FP32` reciprocal (vectorized `vdivq_f32` in 4-group kernels; scalar `1.0f / static_cast<float>(sc)` in 2-group fallback), which is narrowed back to `FP16` (`vcvt_f16_f32` / `vdupq_n_f16`) for vectorized `FP16` multiplication (`vmulq_f16`), round-to-nearest-even (`vcvtnq_s16_f16`), and nibble packing (`vuzpq_u8`, `vorrq_u8`) into 128-bit registers.

2. **In-Register ChaCha20 Encryption & SipHash-2-4-128 Absorption:** A 4-block transposed NEON engine (`chacha20_4blocks_rfc8439`) generates 256 bytes of RFC 8439 ChaCha20 keystream per refill using all 256 bits (k_0, k_1, k_2, k_3) of the session encryption key. For quantized vectors (`q_rope`, `q_nope`, `q_v`, which form **83.3%** of a GQA `INT4_SYM` page), 16B/32B/64B register groups are XOR-encrypted and absorbed into the streaming SipHash-2-4-128 state prior to a single store to the output buffer (falling back to a byte-exact scalar path whenever a partial MAC word is pending or the keystream cursor is unaligned).

3. **L1 Stack-Resident Scale Staging:** Because the wire layout places each section's contiguous `FP16` scale array (`s_rope`, `s_nope`, `s_v`, **16.7%** of payload, 2–4 KB per section on GQA) immediately after that section's quantized data, per-group scales are staged during the vector loop into an L1 stack array (`std::array<at::Half, 8192> stack_scales`, falling back to `std::vector` heap allocation only if a non-standard geometry exceeds 8,192 groups per section) and encrypted + SipHash-absorbed in a contiguous pass at the end of each section without reading back destination memory.

3.3 Transactional Session Keyring, Freshness & Crypto-Shredding

To prevent concurrency hazards across multi-threaded workers, `EphemeralSessionKeyRing` synchronizes all session operations under a re-entrant mutex (`threading.RLock`) and enforces four invariants:

1. **CSPRNG-Salted Domain-Separated HMAC-SHA256 Derivation & In-Place Zeroization:** Each call to `derive_or_get(tenant_id, session_id)` for a new or previously shredded session increments a monotonic generation counter gen, extracts a per-generation key prk = HMAC-SHA256(`deployment_pepper`, `tenant:session:gen` $\parallel$ `os.urandom(32)`) (wiping the 32-byte CSPRNG buffer in a `finally` block), and derives mutable `bytearray(32)`

encryption and MAC keys via labeled domain-separated calls HMAC-SHA256(prk, `"ztcp-cold-page-enc-v1\x01"`) and HMAC-SHA256(prk, `"ztcp-cold-page-mac-v1\x02"`). Calling `crypto_shred_session` or `crypto_shred_tenant` overwrites both active and in-flight reservation `bytearray` buffers with zeros in-place; subsequent re-derivation for the same (`tenant_id`, `session_id`) yields an independent generation (gen + 1) that rejects archived blobs with `KVIntegrityError`.

2. **Atomic Pre-Encryption Nonce Reservation (Burn-Before-Encrypt):** Inside `preflight_validate_egress`, all 96-bit ChaCha20 nonces for a page or batch are validated against the session's used-nonce set and **atomically burned into `_used_chacha_nonces` before releasing the lock and before encryption begins**. Concurrent calls racing on the same nonce or overlapping batch ranges deterministically result in exactly one success and one `NonceReuseError`, and any nonce from a failed mid-flight encode remains permanently burned.

3. **Versioned Page Freshness Contract & Monotonic Watermarks:** Each logical page slot (`model_id_hash`, `layer_id`, `head_group_id`, `page_token_hash`) tracks its highest authenticated `sequence_nonce`. Watermark updates in `commit_egress` and `verify_and_update_freshness` (invoked only after AAD and constant-time MAC verification succeed) use a strict `max()` rule so out-of-order completion of an older in-flight encode can never roll back the slot watermark. Legitimate cache re-reads of the current version (`seq == latest`) succeed, whereas stale-version replays (`seq < latest`) raise `StalePageReplayError`.

4. **Generation-Bound In-Flight Shred Coordination & Linearization:** Every egress reservation (`_StagedNonceReservation`) and ingress context is bound to the integer key `generation`. If `crypto_shred_session` acquires the keyring lock before an operation's `begin_egress_execution`, `commit_egress`, or `finish_ingress` check, the operation detects the revoked generation, zeroizes any output tensor populated inside its execution block (`out_blobs_u8.zero_()` / `k_out.zero_()`), refuses to commit into any newly derived generation, and raises `SessionShreddedError`. Conversely, if `commit_egress` or `finish_ingress` completes under the lock *before* `crypto_shred_session` acquires the lock, the operation is linearized prior to the shred and may return normally after the concurrent shred call returns.

4 Zero-Fork Upstream Hooks & Theseus Guard DREP Integration

ZTCP separates permissive open-source transport hooks (`pr_works/`) from the commercial governance engine (`enterprise_repo/`), aligning cold-page security with the **Theseus Guard Enterprise** DREP (*Detect–Route–Enforce–Prove*) lifecycle [3, 4]:

1. **Pre-Provider Gate & Guard Snapshot Binding:** Incoming requests pass through deterministic pre-checks, **ZT-ADP** Format-Preserving Encryption (FPE), and the **L1 Precedent Decision Cache** bound to `(tenant_id, guard_snapshot_hash)`. When the upstream LLM offloads KV pages via SGLang's `HiCachePageCodec` or LMCache's `DirectBatchBufferSerializer` into `MooncakeCompactStagingRing` views, ZTCP binds the same `(tenant_id, guard_snapshot_hash)` into the 32-byte cold-page AAD header.

2. **PoisonShield Per-Page Recompute Fallback:** If any cold page fails AAD verification, MAC authentication, or freshness checks on ingress, **PoisonShield** blocks writing to hot memory and marks that page slot `False` in the SGLang/LMCache hit bitmap, triggering clean prefix recomputation without crashing the serving worker.

3. **Post-Provider L2/L3 Conviction & Crypto-Shredding:** When post-generation **L2/L3 TG-Sentinel:G1** evaluation or `SessionRiskTracker` convicts a session [3, 4], session keys are zeroized in $O(1)$ time, rendering offloaded cold pages undecipherable.

5 Experimental Evaluation

All benchmarks were executed on a verified 16-core Apple Silicon M3 Max system (12 Performance cores + 4 Efficiency cores, 64 GB Unified Memory, Python 3.12.11, PyTorch 2.10.0, Apple Clang with `-O3 -std=c++17 -march=armv8.5-a+fp16 -ffast-math`) across $N = 5$ independent OS process runs (`bench_cold_page_pipeline.py -runs 5`). All raw per-process results and aggregated statistics are archived in `research/benchmarks/latest_results.json`.

5.1 Synthetic RoPE-Outlier Fidelity & Wire Compression

Rotary Position Embeddings (RoPE) concentrate high-magnitude sinusoidal outliers in the leading 64 dimensions of Key vectors. Tables 1 and 2 evaluate wire compression (including the 32B header and 16B MAC)

and single-page simulated attention cosine similarity on **synthetic KV tensors with structured RoPE outlier injection** (where `generate_realistic_kv_page` always sets `torch.manual_seed(42)` on its target device and `evaluate_attention_fidelity` sets `torch.manual_seed(99)` on `k_ref.device` when sampling the random query vector q). Four methodological details govern these tables:

1. **Synthetic Proxy Scope:** These metrics measure single-page synthetic reconstruction fidelity (using `Qwen/Qwen3.6-35B-A3B` GQA and `DeepSeek-V4.1-Flash` MLA page geometries), not end-to-end LLM perplexity on pretrained model weights.

2. **GQA Output Cosine vs. Single-Buffer MLA Weight Cosine:** For **GQA-Standard** (Table 1, where V is present), `evaluate_attention_fidelity` computes simulated **Attention Output Cosine Similarity** $\cos(\text{softmax}(QK^\top/\sqrt{d})V, \text{softmax}(Q\hat{K}^\top/\sqrt{d})\hat{V})$. For **single-buffer DeepSeek-MLA** (Table 2, where V = None in `bench_cold_page_pipeline.py:88-89, 137-140`), it computes **Softmax Attention Weights Cosine Similarity** $\cos(\text{softmax}(QK^\top/\sqrt{d}), \text{softmax}(Q\hat{K}^\top/\sqrt{d}))$.

3. **Quantizer Arithmetic:** The Python codec is mixed-precision (`FP16` input/output tensors and `FP16` stored scales with `FP32` intermediate group division), whereas the C++17 NEON codec uses `FP16` group abs-max (`vmaxvq_f16`), exact IEEE-754 `FP32` reciprocal division (vectorized `vdivq_f32` / scalar reciprocal) narrowed to `FP16` (`vcvt_f16_f32` / `vdupq_n_f16`), and vectorized `FP16` multiplication/nearest-even rounding (`vmulq_f16`, `vcvtnq_s16_f16`).

4. **Device-Specific RNG Streams:** In Section 1 of the benchmark, when `mps` is available, the `seed=42` KV page is synthesized on `mps` and copied to `cpu` for C++ evaluation; thus `evaluate_attention_fidelity` samples q (`seed=99`) on `mps` for Python and on `cpu` for C++, so small differences between the Python and C++ columns reflect both quantizer rounding and device-specific query RNG streams. Likewise, the C++ GQA cosine in Section 1 (`0.996103`, `mps`-synthesized `seed=42` KV copied to `cpu`) and in Section 4 (`0.995925`, `cpu`-synthesized `seed=42` KV) are both `seed=42` KV pages with `seed=99` CPU queries, differing only by the device RNG used during KV synthesis.

Preserving the 64-D RoPE sub-vector in 8-bit (`PRESERVE_FP8`) matches the simulated GQA attention output fidelity of full `FP16` RoPE preservation (`0.996196` Py / `0.996103` C++ on `mps`-synthesized `seed=42` KV, and `0.995925` C++ on `cpu`-synthesized `seed=42` KV) while saving 22.6% in wire bytes.

Table 1: Synthetic GQA-Standard Fidelity (`Qwen/Qwen3.6-35B-A3B` Profile: `64 tok/page`, `8 KV heads`, `head_dim=128`, `rope_dim=64`, Hot Page = `262,144 B / 256 KiB / 0.262 MB`; Metric = **Attention Output Cosine Similarity**, V Present).

Cold-Page Configuration	Cold Blob	Ratio	Py K Cos	Py Out Cos	C++ Out Cos
INT4_SYM + Quantize All	81,968 B	3.20×	0.99820	0.992378	0.993649
INT4_SYM + Preserve RoPE (FP8)	**98,352 B**	**2.67×**	**0.99984**	**0.996196**	**0.996103**
INT4_SYM + Preserve RoPE (Raw FP16)	127,024 B	2.06×	0.99984	0.996194	0.996088
K8V4_ASYM + Preserve RoPE (FP8)	**114,736 B**	**2.28×**	**0.99999**	**0.996316**	**0.996354**
FP8_ONLY (2x Baseline)	147,504 B	1.78×	0.99999	0.999974	0.999974

Table 2: Synthetic Single-Buffer DeepSeek-MLA Fidelity (`DeepSeek-V4.1-Flash` Profile: `128 tok/page`, `1 latent head`, `head_dim=576`, `rope_dim=64`, Hot Page = `147,456 B / 144 KiB / 0.147 MB`; Metric = **Softmax Attention Weights Cosine Similarity**, V = None).

Cold-Page Configuration	Cold Blob	Ratio	Py K Cos	Py Wt Cos	C++ Wt Cos
INT4_SYM + Quantize All	46,128 B	3.20×	0.99781	0.998846	0.998615
INT4_SYM + Preserve RoPE (FP8)	**50,224 B**	**2.94×**	**0.99905**	**0.999473**	**0.999520**
INT4_SYM + Preserve RoPE (Raw FP16)	57,392 B	2.57×	0.99906	0.999517	0.999521
K8V4_ASYM + Preserve RoPE (FP8)	**82,992 B**	**1.78×**	**0.99999**	**0.999994**	**0.999995**

5.2 Kernel Throughput Tiers & Descriptive Comparisons ($N = 5$ Process Runs)

Table 3 and Figure 1 present the measured performance across $N = 5$ independent OS process runs (×50 iterations per batch tier) for the unified RFC 8439 ChaCha20 + SipHash-2-4-128 kernel, reporting **both hot-byte processing throughput and encoded wire output throughput**.

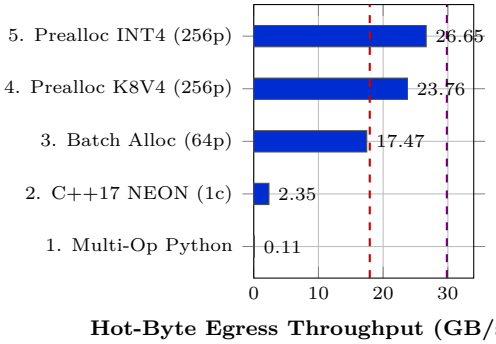


Figure 1: Measured cold-page egress throughput tiers on Apple M3 Max ($N = 5$ independent process runs). Dashed lines denote the 100GbE RoCEv2 Theorem 1 break-even threshold (17.93 GB/s hot, red) and 100GbE full wire-saturation threshold (29.85 GB/s hot, violet).

Incremental Implementation-Tier Comparisons (Not Controlled Causal Ablations): The cumulative **236.05** ± **4.76**× ratio over Tier 1 and the intermediate step ratios in Table 3 are *incremental*

Table 3: Measured Kernel Latency & Throughput on 262,144 B (256 KiB) GQA Page ($N = 5$ Independent Process Runs ×50 Iterations; RFC 8439 ChaCha20 + SipHash-2-4-128 in Tiers 2–5).

Measured Implementation Tier	Egress (μs/p)	Ingress (μs/p)	Hot (GB/s)	Wire (GB/s)	vs. Py	vs. 1c
1. Multi-Op Python (1c, BLAKE2b)	2,322.4 ± 36.2	1,838.3 ± 40.8	0.11 ± 0.00	0.04 ± 0.00	1.0×	0.05×
2. C++17 NEON (1c, INT4)	111.4 ± 0.6	113.1 ± 0.4	2.35 ± 0.01	0.88 ± 0.01	20.9×	1.00×
3. Batch w/ Alloc (64p, INT4)	15.02 ± 0.38	11.39 ± 0.19	17.47 ± 0.43	6.55 ± 0.16	154.8×	7.42×
4. Peak Prealloc CPU (256p, K8V4)	11.04 ± 0.13	—	23.76 ± 0.27	10.40 ± 0.12	210.4×	10.09×
5. Peak Prealloc CPU (256p, INT4)	**9.84 ± 0.17**	**10.15 ± 0.11**	**26.65 ± 0.46**	**10.00 ± 0.17**	**236.1×**	**11.32×**

implementation-tier comparisons across compounded system changes, not controlled single-factor causal ablations isolating register fusion alone: (1) moving from Tier 1 to Tier 2 (**20.85 ± 0.29**×) simultaneously replaces multi-operator Python/PyTorch and `BLAKE2b-128` with compiled C++17 (`-O3 -march=armv8.5-a+fp16`), `FP32`-reciprocal-narrowed-to-`FP16` NEON quantization, 4-block vectorized ChaCha20, `SipHash-2-4-128`, and in-register vector encryption/MAC absorption (with L1 stack scale staging); (2) moving from Tier 2 to Tier 3 (**7.42 ± 0.20**× over 1-core C++) parallelizes a 64-page batch via Grand Central Dispatch (`dispatch_apply_f`) including per-call output tensor allocation and keyring nonce reservation (**17.47 ± 0.43 GB/s** hot egress and **23.02 ± 0.38 GB/s** hot ingress); and (3) moving from Tier 3 to Tier 5 (**1.53**× over Tier 3, **11.32 ± 0.23**× over 1-core C++) simultaneously increases the batch size from 64 to 256 pages and writes into a **caller-preallocated CPU `uint8` tensor buffer** (avoiding per-call output allocation in the loop, without performing hardware RDMA NIC memory registration), reaching **26.65 ± 0.46 GB/s** hot egress (median **26.67 GB/s**, range 26.00–27.09 GB/s) and **25.84 ± 0.28 GB/s** hot ingress.

5.3 Analytical Serial-Egress RDMA Transfer Latency Model

Table 4 evaluates an **analytical serial-egress model** ($T_{\text{total}} = T_{\text{compute}} + T_{\text{wire}}$, not a live NIC-to-NIC RDMA measurement) for offloading a 4,096-token prefix across 64 layers ($4,096$ pages $= 1,073.74$ MB SI / $1,024.0$ MiB raw `BF16`) at an assumed 100GbE RoCEv2 goodput of $BW_{\text{net}} = 11.2$ GB/s (11.2×10^9 B/s). We explicitly separate **analytical latency using empirically measured Apple M3 Max CPU fused compute** (Rows 4a and 5a) from the **hypothetical GPU SRAM (950 GB/s) analytical projection** (Rows 4b and 5b). Combining measured CPU compute (40.31 ± 0.70 ms) with analytical 100GbE wire transfer (35.97 ms), serial ZTCP `INT4` achieves **76.28 ± 0.70 ms** vs. **95.87 ms** for unencrypted raw `BF16` (**1.26 ± 0.01**× speedup, consistent across both the ratio of 5-run mean latencies 95.87/76.28 = 1.2568× and the mean of 2-decimal rounded per-run ratios $[1.25, 1.27, 1.27, 1.24, 1.26]$ in `latest_results.json`), while `K8V4` achieves **87.15 ± 0.53 ms** (**1.10 ± 0.01**× speedup).

Table 4: Analytical Serial-Egress 4,096-Token Prefix Transfer Model across 64 Layers (11.2 GB/s SI Goodput), Separating Measured M3 Max CPU Compute from Projected GPU SRAM (950 GB/s).

Analytical Serial Pipeline Path	Wire (MB / MiB)	Compute	Wire	Total	Speedup
1. Unencrypted Raw BF16 Baseline	1,073.7 / 1,024.0	0.00 ms	95.87 ms	95.87 ms	1.00×
2. Discrete Host Copy (28 GB/s) + K8V4	470.0 / 448.2	38.35 ms	41.96 ms	80.31 ms	1.19×
3. Unencrypted Raw FP8 (Hot Quant.)	536.9 / 512.0	0.00 ms	47.93 ms	47.93 ms	2.00×
4a. ZTCP K8V4 (Measured M3 Max CPU)	**470.0 / 448.2**	**45.19 ms**	**41.96 ms**	**87.15 ms**	**1.10×**
5a. ZTCP INT4 (Measured M3 Max CPU)	**402.8 / 384.2**	**40.31 ms**	**35.97 ms**	**76.28 ms**	**1.26×**
4b. ZTCP K8V4 (Projected GPU @ 950 GB/s)	470.0 / 448.2	1.13 ms	41.96 ms	43.09 ms	2.22×
5b. ZTCP INT4 (Projected GPU @ 950 GB/s)	402.8 / 384.2	1.13 ms	35.97 ms	37.10 ms	2.58×

5.4 Security Verification Matrix, Protocol Contracts & Limitations

Table 5 summarizes the automated security verification suite bundled in `bench_cold_page_pipeline.py` (`run_security_verification`), specifying the exact exercised backend for each test group and labeling local in-memory byte/bit mutations as simulated RDMA tampering. (In-flight egress and ingress shred checks across all four codec paths `py_single`, `cpp_single`, `cpp_batch`, and `cpp_peak` are verified in the separate local regression suite `verify_package2.py`, section `CODE-003`; overlapping-batch nonce checks are verified in section `B6-2`, distinct from the bundled benchmark.)

Table 5: Bundled Security, Freshness & Concurrency Matrix by Backend (`bench_cold_page_pipeline.py`).

Test ID	Backend Path	Verified Scenario	Observed Outcome	Status
3.1–3.4	Py Ref (`BLAKE2b`)	Round-Trip, AAD, Sim. 1b Flip, Shred	`CrossTenant` / `KVIntegrityError`	**PASS**
3.5	C++ Peak (`SipHash`)	Native Peak Decode & Cache Re-Read	Exact Decode (×2)	**PASS**
3.6–3.8	C++ Peak (`SipHash`)	Sim. Scale, 16B Swap & V Bit Tamper	`KVIntegrityError`	**PASS**
3.9, 3.11	C++ Peak (`SipHash`)	4× AAD Drift & Post-Shred Re-Derive	`CrossTenant` / `KVIntegrityError`	**PASS**
3.10a–c	C++ Peak + Py Nonce	Nonce Helper, Peak Dup Nonce & Stale Replay	`NonceReuse` / `StalePageReplay`	**PASS**
3.12	C++ Peak & Single	Peak Same-Nonce Race; 1p Burn & Watermark	`NonceReuse` / `StalePageReplay`	**PASS**

Implementation Scope, Protocol Contracts & Open Caveats:

1. **In-Process Keyring Concurrency, Linearization & Buffer Ownership Contracts (`CODE-003`):** (a) *Single-process lock scope:* Thread safety, atomic nonce burning, and monotonic watermarks are enforced in-memory within a single Python process via `threading.RLock` (verified by bounded multi-threaded tests). (b) *Pre-shred linearization:* Operations whose locked `commit_egress` or `finish_ingress` check completes *before* `crypto_shred_session` acquires the lock are linearized prior to the shred and may return normally after the concurrent shred call returns; operations reaching `begin_egress_execution`, `commit_egress`, or `finish_ingress` *after* shred zeroize outputs populated in their execution block (`out_blobs_u8.zero_()` / `k_out.zero_()`) and raise `SessionShreddedError`. (c) *Peak buffer ownership & exception invalidation:* The caller of `encode_cold_pages_peak_into` (and `decode_cold_pages_peak_into`) must exclusively own the preallocated output buffer, read/publish it only after a normal return, and discard it

as invalid on *any* exception; preflight errors in `preflight_validate_egress` and input/output buffer-overlap checks occur before the buffer-zeroing `try...except` block and do not clear prior buffer contents. (d) *User-space memory boundary:* While mutable `bytearray(32)` key buffers and per-call mutable snapshots are wiped in-place, immutable Python `bytes`, native C++ stack/integer key copies ($k_0..k_3$, $mk_0..mk_1$), CPython GC temporaries, and OS swap/core-dumps remain outside any hardware memory-erasure guarantee.

2. **Wire Version, Backend MAC Non-Interoperability, Binding Defaults & Verified Build Scope:** The 32B header retains `FORMAT_VERSION = 1`. The Python reference codec uses 128-bit `BLAKE2b` whereas the native C++17 codec uses 128-bit `SipHash-2-4-128`; each backend decodes its own blobs. Low-level pybind11 entrypoints keep default `k2=0, k3=0` arguments for legacy signature compatibility while `NativeCppFusedColdPageCodec` always passes all 256 bits ($k_0..k_3$). Finally, `native_fused_codec.py` passes `-march=armv8.5-a+fp16` unconditionally on POSIX builds (verified on Apple Silicon `arm64`), and `-reaggregate-only` updates its target JSON file in place unless `-output-json` is directed to a scratch copy.

6 Conclusion & Artifact Availability

ZTCP demonstrates that confidential KV-cache offloading over RDMA can achieve net latency reductions over unencrypted raw transfers whenever fused throughput exceeds the Theorem 1 break-even bound. By combining RoPE-preserving asymmetric quantization with 256-bit RFC 8439 ChaCha20 encryption, 128-bit SipHash-2-4-128 authentication, and transactional session-state management, ZTCP shrinks cold-page wire footprints by 2.67×–2.94×, reaches 26.65 ± 0.46 GB/s hot (10.00 ± 0.17 GB/s wire) egress and 25.84 ± 0.28 GB/s hot ingress throughput on an Apple M3 Max CPU ($1.26 \pm 0.01\times$ faster in our analytical serial-egress 100GbE model using measured CPU compute time, and 2.22×–2.58× under projected GPU SRAM execution), and enforces verifiable multi-tenant isolation, freshness, and in-process crypto-shredding.

Code & Reproducible Benchmarks: The open-source C++17 ARMv8.5-A FP16 NEON codec, 32-byte AAD wire specification, zero-copy upstream plugin hooks (`sglang`, `lmcache`, `mooncake`), and reproducible 5-run benchmark artifacts are publicly available at https://github.com/Theseus-Guard/ZTCP and archived under CERN/Zenodo DOI 10.5281/zenodo.23527048.

References

[1] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[2] Hakan Damar. Advanced electronic warfare systems for hybrid intelligence wars (Hibrit İstihbarat Savaşları İçin Gelişmiş Elektronik Harp Sistemleri). Master's thesis, Bahçeşehir University, Graduate Education Institute, Information Technology Master's Program, Istanbul, Türkiye, 2023. Supervisor: Dr. Ahmet Naci Ünal. Covers multi-agency HP-CaaS isolation, low-precision approximate computing accelerators, and post-quantum cryptography (CRYSTALS-Kyber / Dilithium) against harvest-now-decrypt-later threats.

[3] Hakan Damar. Systems and methods for stateful, tenant-specific governance, selective inference mediation, and runtime enforcement of artificial intelligence agent executions, 2026. Theseus Guard, Inc., U.S. Provisional Patent Specification v1.0 & Enterprise Architecture (DREP Lifecycle, L1 Precedent Decision Cache, Zero-Trust AI Data Plane ZT-ADP, and Governance Snapshot Binding).

[4] Hakan Damar and Theseus Guard Research. TG-Sentinel:G1: Unified foundation AI governance and technical risk analysis model family (Fast, Max, Ultra), 2026. Theseus Guard Technical Specification v1.0.5.

[5] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[7] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.

[8] National Institute of Standards and Technology (NIST). Module-lattice-based key-encapsulation mechanism standard (FIPS 203) and digital signature standard (FIPS 204). Technical report, U.S. Department of Commerce, August 2024.

[9] NVIDIA Corporation. NVFP4 cold-page KV-cache compression in TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM/blob/main/examples/kv_cache_compression/nvfp4_cold_page.md, 2026.

[10] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[11] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.